

Project Proposal

Group 1: Alice Guth, Maddie Creed, DuBose Tuller, David Yoder, Katherine Hu, Ann Zhu,
Wendy Xu, Ally Smith
February 20, 2025

1. Introduction: Maddie

The name of our project is Grade Gator. Grade Gator aims to create an online grading tool for professors and students with the flexibility to support manual and automated grading for standards-based and traditional-based grading preferences. The motivation for this project comes out of necessity, as on-market alternatives fail to adequately address the needs of the Davidson Math & Computer Science department faculty and students. Our product will meet the needs of the department, specifically by including the ability to use an autograder to evaluate coding assignments, integrate standards-based grading, provide grade book features, and create novel feedback options such as in-line comments. Our customer is the Davidson Math & Computer Science department, but our product is generally useful for all higher education institutions. The users of this product are professors, TAs, students, and anyone else involved in the process of grading and submitting assignments. The interface will be different based on the user's role in the course, including a separate grader/professor view and student view. We will accomplish this using the scrum development process, which is based on Agile principles, and is composed of development sprints.

2. Novelty: Katherine

In our case, the software programs for comparison are Gradescope and Moodle. While Gradescope and Moodle offer useful functionalities for handling assignments and grading, our customers—professors in the Math & CS department—are dissatisfied with certain features. During our first developer-customer meeting, we discussed areas for improvement and customization specific to our department. These enhancements not only address faculty concerns but also introduce novel features compared to Gradescope.

Enhancing Student Experience

- Students should be able to view the rubric before submitting their assignments.
- Students would need to click through every feedback comment to acknowledge them before viewing their scores.

Improving Grading and Grade Management

- The gradebook should automatically calculate final grades for both students and professors.

- Standard-based grading should be incorporated to better accommodate courses at Davidson.
- Add visualizations to show where most of the points were deducted
- Allow inline comments when giving feedback.

Course and Assignment Organization

- The roster sorting options should be expanded. Currently, Gradescope sorts names alphabetically by first name, while Moodle sorts them by last name. Our software should allow multiple sorting options.
- Courses should be grouped by name rather than by the time they were created.
- Assignments should be configurable so they can be released to specific students instead of the entire class.

3. Customer Need: Alice

Who is the primary customer outside the team?

- The primary customer for our product Dr. Lim and Dr. Heyer at Davidson College.

Who are the secondary stakeholders?

- The secondary customers include the Mathematics and Computer Science department at Davidson, and more broadly, students and professors at universities across the country.

What do the stakeholders want? Why?

- The stakeholders want a web-based assignment manager that includes code autograding functionality and allows them to provide assignments to their students, give feedback on said assignments, and organize student responses in an efficient and effective manner. The stakeholders have frustrations with the product they currently use to provide this functionality and want to create a platform to serve the same purpose but that is tailored to their needs and easier to use.

What is their desired overall experience?

- The stakeholders want to be able to set up assignments for their classes on this platform, distribute them to their students, have an autograder check the work and then provide students with their grade and additional feedback in an effective manner.

3.1 User Requirements

1. As an instructor I want to flip between traditional and standards based grading on different assignments so that I can provide students with appropriate feedback and opportunities to correct mistakes
2. As a student, I want to be able to see the feedback and rubric for each assignment so that I can understand how I am being graded and can get clarification on areas of confusion.
3. As an instructor I want to upload assignments with unique accommodations for each student so that I can meet the needs of each student.
4. As an instructor and student, I want to organize my course dashboard based on criteria like most recent or by subject so that I can access courses efficiently when I need to.
5. As an instructor I want to see which questions were missed by the most students so that I can clear up any widespread confusion in class.

3.2 Acceptance Tests

1. Given I am getting ready to grade an assignment, when I tell the platform I want to grade using standards based grading then it will determine how many standards each student achieved on the assignment.
2. Given I am preparing to complete an assignment, when I open the assignment and navigate through the questions, then the rubric for each question should also appear alongside the question.
3. Given I am preparing to release an assignment to a class, when I am on the settings page to set the time allowed for the assignment and when it is due, then each student should be automatically given whatever additional time or other accommodations they require.
4. Given I am getting ready to complete an assignment, when I open the homepage of the software, then I should be able to find the course I'm looking for quickly.
5. Given I am reviewing my student's work on a recent assignment, when I navigate to the results of the assignment then I will be able to see the distribution of who missed each question.

4. Project Goals: Ann and Wendy

4.1 Customer Problems and Benefits

- What customer problem have you chosen to address?

In general, we aim to provide a more efficient grading platform for professors from the Math&CS department. This includes some basic functionalities a grading tool should have, including

1. Supporting various submission formats, including Python, Java, and C files, as well as PDFs for written responses
2. Enhancing auto-grading functionality by allowing instructors to define test cases and packages, while ensuring clearer and more detailed error messages for students

3. Implementing flexible due dates that accommodate individual student needs across all assignments
 4. Improving feedback visibility by ensuring students can easily locate and acknowledge instructor comments before viewing their scores
 5. Facilitating standard-based grading to help students track their progress and understand how their grades are calculated
 6. Providing better sorting and roster management, addressing faculty concerns about Gradescope's limited sorting functionality
- In implementation-free terms, what user benefit will the system provide?
Our system will provide a more flexible, transparent, and efficient grading experience for both instructors and students. Professors will gain better control over assignment submissions, grading workflows, and accommodations, while students will receive clearer feedback and improved insights into their academic performance.
 - How will the benefit support the customer's desired overall experience?
By addressing the inefficiencies of current grading platforms, our solution will:
 1. Greater flexibility and ease of use for professors – The system will support customizable grading workflows, allowing instructors to adapt deadlines, accommodate individual student needs, and manage submissions efficiently.
 2. Clearer feedback and engagement – Students will have better visibility into instructor comments, making it easier to review and engage with feedback before viewing their grades.
 3. Enhanced transparency in grading – Standard-based grading and visual insights will help students better understand their performance and how their grades are determined.
 4. Increase system compatibility by aligning features with Moodle and supporting instructor-defined grading criteria.

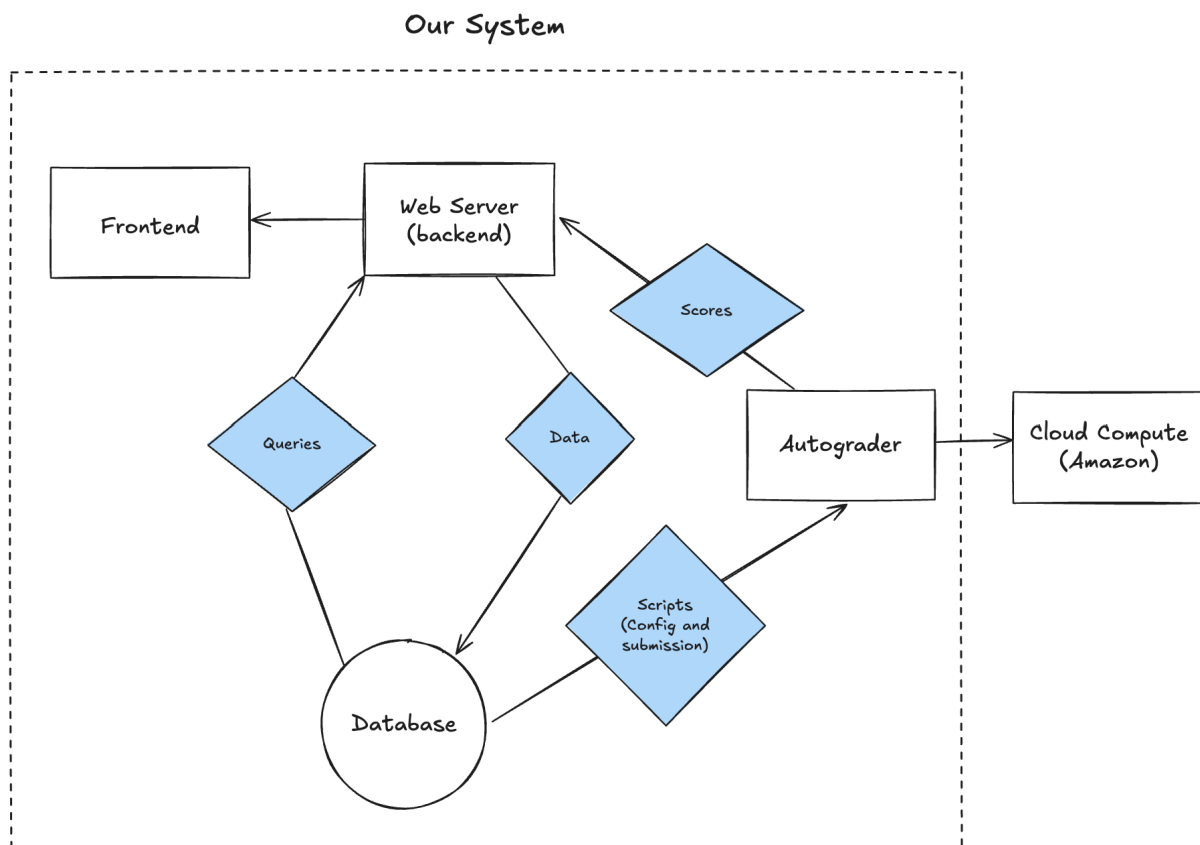
4.2 Measure of Success

- Who outside the team have you tested the idea on?
We have gathered feedback from professors in the Computer Science department (Dr. Lim, Dr. Williams, Dr. R, and Dr. Pulaj), who identified key pain points in current grading tools and suggested improvements, such as better rubric visibility, improved roster sorting, and enhanced auto-grader functionality.
- How will you know whether the customer got their desired benefits?
We will release the test versions of our program to instructors and students and collect feedback on their experience using the program. We will ask them to complete a survey that has grading portions on each functionality of the program and comment sections so that they can rate what functionalities should be improved and any additional functionalities they want us to include.

- What are your customer-centric measures of success?
Our customer-centric measures of success include: whether the interface is user-friendly and clear and whether the functionalities satisfy users' needs. These are the two measures we would focus on, attempting to optimize.

5. System Description: DuBose and David

Draw a block diagram to show how the proposed system will interact with external services, databases, etc. Clearly mark the boundaries of the System.



What are the main elements of the proposed system?

- Database: store courses, students/instructors, scores, file uploads, etc.
- Web server: handle user input, query and add to database, call upon the autograder
- Autograder: take instructor configuration and student code submission, execute unit tests on student code according to the config, and send a score report to the web server.

- Frontend: display contents to the user in a convenient and useful way

6. Solution Approach: DuBose and David

Briefly describe how the system will work.

We will have a single web server that handles the core functionality of the service. The web server will have three core functionalities, with others likely being added as the project progresses. It will communicate with a database of user and course data, call the autograder, and send results to the frontend.

What technologies (platform, tools, libraries, programming languages) will you use and why?

- React for frontend
- shadcn/ui library
- Python for backend
- SQL for a relational database
- Amazon Web Services for cloud compute instances
- Docker for creating an autograder container

How will you test and measure the adequacy of your test strategy?

We will find/construct a test dataset of student submissions across multiple courses. Autograding files from previous 221 classes. We will also place an emphasis in adversarial testing, making sure that we cannot, for example, ‘break out’ of the containerized compute instances. If feasible, we will add an automated test suite to our GitHub repositories.

7. Project Management: Katherine

We chose to use Scrum as our development method. The primary reason is that our customer required it, but we all agreed with the decision. Compared to XP or Scrum + XP, XP requires constant interaction with customers and is well-suited for handling frequently changing requirements. However, in our case, it is uncertain whether professors will be available for regular meetings, and their needs are unlikely to change during this period. Therefore, we chose Scrum to ensure stable updates and to communicate with customers on a biweekly basis.

Proposal - Report 1: Break down the system into components, figure out the tech stack, start designing the interface, and have the basic functionality of Gradescope in terms of creating

courses, adding students, assigning roles (instructors, TA, students), allowing code/pdf submissions

Report1 - Report 2: Work on grading: allow instructors to upload tests, provide the option of standard-based grading, provide better error handling, allow inline comment editing, support Python, Java, and C in the system

Report 2 - Final: Add additional functionality such as grade book, visualizations, sorting...

8. Team Management: Everyone

Roles:

Scrum Master: DuBose Tuller

Product Owner: Alice Guth

Software Developers: Alice Guth, Maddie Creed, DuBose Tuller, David Yoder, Katherine Hu, Ann Zhu, Wendy Xu, Ally Smith

Scheduling:

- Weekly meeting in-person meeting Wednesdays at 4:30, every other day check ins
- Virtual presentation of software to customer every two weeks, additional in-person meetings as needed

8.3: Background

Maddie has experience with python, c, and java languages as well as human-computer interaction and user-centered design principles.

David has experience with a wide range of javascript frameworks, consisting of React, Svelte, vanilla javascript, Three.js, D3.js, React three fiber, Tailwind CSS, vanilla CSS. Other languages include Python and Java.

Ally has experience with python, C, Java, JavaScript, HTML, and CSS. Mostly focuses on UX and HCI through courses like Data Vis and HCI.

Wendy has experience programming with python, java, c, swift, java script languages, experience working in research groups that involve teamwork in STEM fields, experience with some visual designing/UI design.

Katherine has experience in web development and experience with frameworks such as React and Next.js. She also has experience in mobile development with React Native and Expo. She has some experience with Firebase Database and Firebase Authenticator. Programming languages she's familiar with are Python, Java, Typescript/ JavaScript, HTML/CSS, C#, SQL.

DuBose has experience with Python and some experience with relational databases from the Database Systems class. He has limited experience with cloud computing (Google) and Docker for containerized computation. He is experienced with git and GitHub

Ann is familiar with Python, Java, and C languages. She also has experience in backend mobile app development using FastAPI, PostgreSQL, and Firebase.

Alice has experience developing a mobile virtual reality app using Unity and Xcode, and has experience with Python, Java, and C.

9. Constraints and Risks: Ally

Are there any social, ethical, policy, or legal constraints?

- **FERPA Compliance:** Student grades and submissions must be protected under the Family Educational Rights and Privacy Act → Proper data security and privacy.
- **Plagiarism & Academic Integrity:** AI-assisted grading or similarity detection, ensure it aligns with university policies on academic integrity.
- **User Consent & Data Ownership:** Clearly define who owns the submitted data and whether students or instructors must consent to their data being stored or used.
- **Accessibility Compliance:** Follow WCAG (Web Content Accessibility Guidelines) to be usable by all students, including those with disabilities.
- **Copyright & Patented Designs:** Ensure we have novelty in our design and do not copy previously existing platforms. Create new features and have unique UI designs to comply with ethical and legal constraints.

Will you have access to the data, services, and resources you need?

- **University API Access:** Integrate with university systems (e.g., LMS like Canvas, Blackboard), check if they provide APIs and if we have permission to use them.
- **Cloud Storage & Compute Power:** If handling large file uploads or automated grading, ensure we have enough storage and processing resources.
- **Funding or Hosting Support:** Host and maintain system long term
 - To determine costs and needed computing power, we would need data on how many classes and how many students will be using the platform. We also need to know how many of those classes will upload code and use the autograder.
 - We will host the server on AWS.
 - We will not need a lot of computing power during the semester because we will only be testing the system, so the cost should be little.
- **Authentication Services:** May need university SSO (Single Sign-On) integration for authentication, which could require administrative approval.
 - We do not need to focus on this yet, but we can work with T&I when the time comes.

Is there anything else you might need?

Focus on now:

- **Time Constraints:** Figure out tasks that need to be done before the end of semester versus end of summer.
- **System Scalability:** Can the system handle many concurrent users during peak submission times?

Can focus on later:

- **User Testing & Feedback:** Real students, professors, and graders to test the system and provide feedback.
- **Security Risks:** Ensure data encryption, secure logins, and protection against common threats like SQL injection or cross-site scripting (XSS).

10. GitHub: DuBose

We have created a GitHub Organization to store all of our code repositories and documentation. We will use GitHub projects to assign tasks and link them to user stories and/or pull requests.

- <https://github.com/CSC-312-Grades>